

XQDot4Z: Quantized Dot Products on a RISC-V Processor

Steve Andy Ildefonso Santos

Abstract—Storing neural-network weights at four bits saves memory, but a processor must still take them apart, and affine quantization adds a cost that is easy to overlook: a weight code is not a weight but an offset from a zero point that has to be subtracted first. This work asks whether that subtraction is worth building into the dot-product instruction. A fused four-term instruction is added to Kuntur, a teaching five-stage RISC-V core, and compared against three points of reference under one frozen code-generation policy: software multiplication without a multiplier, scalar multiply, and a packed dot product whose correction is factored so the activation sum is computed once per group and reused across rows. A registered campaign of 2280 simulated cases, all passing in two independent simulators, yields a map of conditions rather than a single speedup. Fusion reduces cycles against both alternatives, but the margin narrows as the row count grows, because the factored baseline spreads its correction over more rows, and vanishes when the zero point is zero, where that baseline omits the correction entirely. Where the zero point varies per row the advantage is paid for in code size: the zero point travels as an immediate, so each distinct value needs its own row body, and the fused variant occupies 516 instruction words against 72. Part of the raw gap is loop control rather than arithmetic, separated here by measuring a structural twin. These are simulated cycles, not physical time, energy or area.

Index Terms—quantized inference, low-precision arithmetic, instruction-set extensions, hardware verification

I. INTRODUCTION

A neural-network layer is, at bottom, a great many dot products between weights and activations. Quantization lets both sides be held as small integers with a scale factor [1], and the saving in storage is dramatic: eight four-bit weights fit in one 32-bit word. But that is a saving in memory, not in work. The processor still has to pull the word apart before it can multiply anything, and the unpacking is not free.

Affine quantization adds a second cost that is easier to overlook. A four-bit weight code is not the weight. It is an offset from a zero point shared by a group of weights, so what the arithmetic actually needs is the code minus that zero point. One subtraction is nothing. Four subtractions on every group of every dot product in every row of a layer is not nothing.

This work asks a deliberately narrow question about that subtraction: is it worth building into the instruction that computes the dot product? Such an instruction is added to Kuntur, a teaching five-stage RISC-V core, and measured against three points of reference that handle the same arithmetic in different ways. The goal is not an accelerator, and not a complete inference stack. It is a map of the conditions under which fusing the correction pays for itself, including the conditions under which it plainly does not.

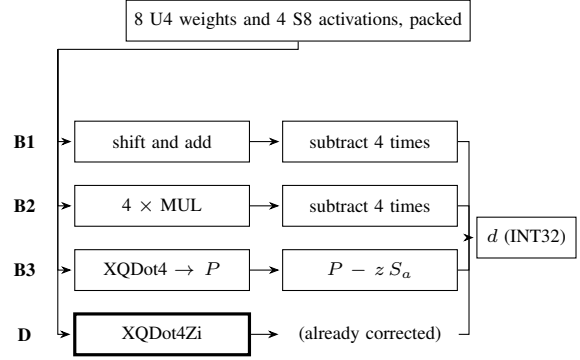


Fig. 1. Work each variant performs on one group of four weights. B1–B3 are points of reference; D is the design under study. All four read the same packed data, so the difference is not in the product but in who pays for the zero-point correction: B1 and B2 subtract it term by term, B3 factors it and reuses S_a across the N rows, and D absorbs it inside the instruction itself.

II. BACKGROUND AND RELATED WORK

Under affine quantization an integer code q stands for $r = s(q - z)$, with $s > 0$ a scale and z a zero point. Taking activations whose zero point is zero, the kernel under study is

$$d = \sum_{i=0}^3 (q_{w,i} - z_w) q_{a,i}. \quad (1)$$

Weights are stored as U4, unsigned four-bit integers in $[0, 15]$, and activations as S8, signed eight-bit integers in $[-128, 127]$. The result is an INT32 subtotal in $[-7680, 7680]$; scales, bias and the accumulation of other subtotals stay outside the operation.

Read literally, (1) costs four subtractions before any product is formed. It can be rearranged, however, into

$$d = \sum_i q_{w,i} q_{a,i} - z_w \sum_i q_{a,i}. \quad (2)$$

The first term is an ordinary dot product over the raw codes. The second is the zero point times the sum of the activations, and that sum does not depend on the weights at all. In a matrix–vector product the same activations meet every row, so it can be computed once and reused across all N of them. This is precisely why a fair comparison cannot simply count the subtractions in (1): a competent implementation never pays them.

The choice of where the zero point lives is not a detail of this design. Surveys of quantization treat symmetric and asymmetric schemes as a standing trade-off: forcing the zero point to zero removes the correction entirely and is cheaper

in hardware, while keeping it uses the integer range better on skewed distributions [2]. Weights are commonly quantized symmetrically for exactly that reason, which is what makes the asymmetric weight case worth pricing rather than assuming. The format studied here, four-bit weights against eight-bit activations, is the one production systems for quantized serving have converged on [3].

Low-bit instruction-set extensions on RISC-V are an established line of work. XpulpNN adds nibble and crumb SIMD instructions to a microcontroller-class core and Dustin carries flexible bit precision across a parallel cluster [4], [5]; Sparq extends the vector ISA with a multiply-shift-accumulate instruction for sub-byte inference [6]. Low-bit dot products are therefore not a new idea and are not claimed as one here. What none of that work isolates is the cost of the zero-point correction itself, because the correction is bundled into a larger kernel where it cannot be separated from packing, from vector width, or from the surrounding loop.

The question examined here is narrower and, as a result, answerable: given that a packed dot product already exists, does the zero-point correction belong inside the instruction or outside it?

III. RESEARCH QUESTION AND SCOPE

Under what conditions does fusing zero-point correction into a U4-by-S8 dot product reduce execution cost compared with scalar multiplication and with factored correction, accounting for activation reuse and for the cost of delivering the zero point?

Scope is one core, batch size one, and small dot-product and matrix-vector kernels. Cost means cycles of a simulated in-order pipeline, reported alongside instruction counts and code size. Synthesis, area, frequency and energy belong to a later stage with a different methodology, and nothing measured here is extrapolated to them.

One distinction runs through everything that follows. A zero point may be constant across a layer (ZC), vary per row but be known when the code is generated (ZS), or be read from memory at run time (ZR). The instruction proposed here carries the zero point as a four-bit immediate, which covers the first two regimes and not the third. ZR is therefore declared out of scope rather than estimated, since an immediate cannot stand in for a runtime value at no cost.

IV. METHOD

A. The Four Variants

Four implementations produce the same outputs from the same packed data. Three are points of reference, named with a leading B for baseline; the fourth, D, is the design under test. Table I states what separates them and Fig. 1 shows what each one does with a single group of four weights.

B1 has no multiplier at all and builds each product by bounded masked bit decomposition. It is context rather than a claim about optimal software: it answers what this operation costs on a core with no multiplication hardware, and its implementation is frozen before measurement so that it cannot

TABLE I
THE FOUR VARIANTS. B MARKS A POINT OF REFERENCE, D THE DESIGN UNDER TEST. ALL FOUR READ IDENTICAL PACKED DATA AND SHARE ONE CODE-GENERATION POLICY, SO WHAT DIFFERS BETWEEN THEM IS THE ARITHMETIC, NOT THE SURROUNDING CODE.

	Multiplier	Dot product	Correction of z
B1	none	bit decomposition	per term, no multiply
B2	scalar MUL	one MUL per weight	per term
B3	scalar MUL	packed, four lanes	factored, reused per row
D	not needed	packed, four lanes	fused into the instruction

be tuned after seeing results. B2 is the straightforward version with a scalar multiply.

B3 is the serious competitor. It runs a packed four-lane unit over the raw codes and applies the correction in the factored form of (2), computing the activation sum once per group and reusing it across rows. D uses a four-lane unit that subtracts the zero point inside each lane, so it yields the corrected subtotal directly and never forms an activation sum at all.

The comparison that answers the question is D against B3. B1 and B2 bound the space around it; they do not settle it.

B. Instruction Encoding

Both optional instructions are R-type and live in custom opcode space, so neither reuses a reserved standard encoding. Written in the notation the base integer instructions use, where `add rd, rs1, rs2` is $X[rd] = X[rs1] + X[rs2]$ and `lw` is $X[rd] = M[X[rs1] + \text{signext}(imm)]$, the two additions read

$$X[rd] = \sum_{i=0}^3 (U4(X[rs1], i+4h) - z) \cdot S8(X[rs2], i) \quad (3)$$

for `xqdot4zi rd, rs1, rs2, z, h`, and

$$X[rd] = \sum_{i=0}^3 U4(X[rs1], i+4h) \cdot S8(X[rs2], i) \quad (4)$$

for `xqdot4 rd, rs1, rs2, h`. Here $U4(w, j)$ is the j -th unsigned four-bit field of a word and $S8(a, i)$ the i -th signed byte, h selects which half of the eight weights is used, and the result is sign-extended to 32 bits. Neither touches memory, neither accumulates into the previous value of rd , and $rd = x0$ discards the result as usual.

The whole contribution is the $-z$ in (3). Everything measured in this paper follows from where that subtraction lives, and from the fact that z is an immediate: Fig. 2 shows it occupying bits 31:28 of the instruction word. That placement is what makes the correction free at run time, and equally what forces a separate row body per distinct zero point.

XQDot4Zi admits 32 combinations of z and h ; every other `funct7` or `funct3` pattern in custom-0 is rejected rather than ignored, and XQDot4 admits only two. Rejecting unimplemented patterns matters more than it sounds: a core that quietly executed them would make the negative controls meaningless.

C. Platform and Verification

The original teaching core is preserved and all development happens in an independent clone, so the baseline cannot drift

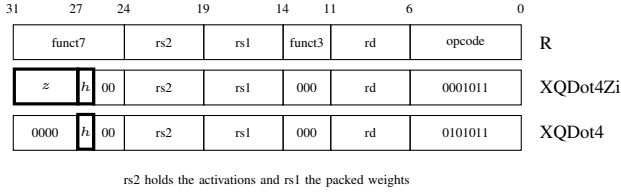


Fig. 2. Format of the two added instructions against the base ISA’s R-type. Both are ordinary R-type words in custom opcode space and reuse no reserved standard encoding. What separates them fits inside funct7: XQDot4Zi spends four of those bits on the zero point z , which is exactly why its correction is free at run time and why its code must be specialized, while XQDot4 leaves them zero and corrects nothing. The h bit selects which half of the eight weights is used.

under the experiment. Each piece is verified on its own before being combined.

The isolated XQDot4Z unit passes 556 734 comparisons per simulator against a Python reference using unbounded integers, together with eight negative controls and four injected mutations that the tests are required to detect; lint needs no waivers. Exhaustiveness covers single-term tuples only. Folding it into the pipeline as the immediate-form XQDot4Zi adds 25 directed programs across two simulators, comparing retirement, writeback, stores and operands, and detecting two further mutations.

The optional scalar MUL used by B2 is independent of that work and returns the low 32 bits of the product [7]; it implements neither the full M extension nor Zmmul. It passes 86 704 vectors per simulator and 49 programs over four configurations. The packed unit behind B3, which omits the zero-point subtraction, passes 67 910 vectors per simulator; XQDot4 integration passes 123 programs in both simulators across all eight configurations of the three optional instructions.

Every milestone is pinned by SHA-256 in a state file, every run is repeated to show that it is deterministic, and mutations are injected deliberately to prove the tests can fail. Two independent simulators, Icarus Verilog and Verilator, must agree exactly on every counter before a case is recorded as passing.

D. Making the Comparison Fair

A comparison between an instruction and its absence is easy to rig, so the degrees of freedom are removed before any measurement. Unrolling, register allocation, scheduling and correction reuse are fixed by one written policy shared by all four variants, and that policy, together with the generators and tests that implement it, is frozen before the campaign runs. The policy explicitly permits omitting work that a particular zero point makes unnecessary, which is how B3 comes to drop its correction entirely when the zero point is zero.

All four variants are assembled without compressed instructions [8], so a code-size comparison counts the same kind of word in every case rather than rewarding whichever variant happens to compress better.

The grid, the seeds, the zero-point profiles and the reporting rules are registered in advance, and the registration pins the hypotheses by hash so that they cannot be edited afterwards. Regimes are never pooled into a single number, and every planned case is retained, failures included, because a case that is merely absent would make an incomplete campaign look finished.

One subtlety deserves its own mechanism. Because D carries the zero point as an immediate, a zero point that varies per row forces it to emit one row body per row, while B3 can keep a loop. Part of any measured gap is therefore loop control rather than correction. To separate the two, each variant is also measured as a structural twin: the same loop body replicated without the back edge. Subtracting the twin prices loop control exactly, so what remains is attributable to the arithmetic.

E. Reproducibility

Every milestone is pinned by SHA-256 in a state file that names the evidence it rests on, and the acceptance gate re-derives those hashes rather than trusting the claim. Each verification runs twice and the two runs are compared field by field, so a result that depended on timing or on ordering would not survive. Mutations are injected into the units, the harness and the registration mechanism itself, and a mutation that no test detects is treated as a missing test. The registration is a chain in which each link pins the bytes of the one before it, so a design that changed after measurement cannot be presented as though it had been fixed in advance.

The whole record is offline and self-contained: no network, no shell escape, and every planned case retained with its terminal status, failures included.

V. RESULTS AND DISCUSSION

The registered campaign ran 2280 cases, all passing in both simulators with none missing. Cycles are identical across the ten seeds in all 228 combinations of variant, shape and setting, and across the sixteen phases of the varying-zero-point profile, while the outputs differ in every one. That is a statement about this kernel and this data layout, not a proof of independence from arbitrary data, and it is explicitly not used to justify running a smaller grid.

Fig. 3 and Table II carry the central result. The advantage of fusion over B3 falls as the row count grows in every regime where B3 has a correction to amortize, which is the trend the work expected. At $z=0$ there is no advantage whatsoever: B3 omits both the activation sum and the correction, and the two variants coincide cycle for cycle. At $N=1$ a varying zero point is degenerate, since a single row cannot have a zero point that varies, so those columns necessarily agree.

Table III breaks the gap down. Every case in the campaign satisfies the identity that cycles equal retired kernel instructions plus the pipeline depth, plus load-use stalls, plus twice the taken-control flushes, and the check recomputes it rather than reading a recorded total. Applied to sixteen rows under a varying zero point, it shows the 164-cycle difference between

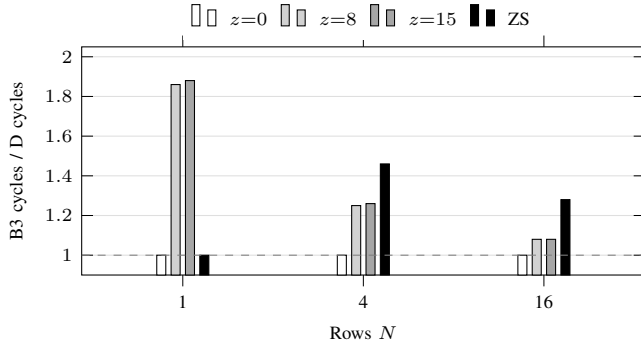


Fig. 3. Advantage of fusion by regime and row count. The dashed line at 1.0 marks the absence of any advantage. The first three groups are constant zero points; ZS makes them vary per row. The advantage narrows as N grows because B3 amortizes its correction over more rows, and at $z=0$ there is none at all because B3 omits it. At $N=1$ a varying zero point is degenerate: a single row cannot have a z that varies. Run 20260911T040655320069Z.

TABLE II

B3 CYCLES DIVIDED BY D CYCLES, BY REGIME AND ROW COUNT. A VALUE OF 1.00 MEANS THE TWO VARIANTS COST EXACTLY THE SAME. RUN 20260911T040655320069Z, 2280 CASES; REGIMES ARE NEVER POOLED.

N	ZC $z=0$	ZC $z=8$	ZC $z=15$	ZS
1	1.00	1.86	1.88	1.00
4	1.00	1.25	1.26	1.46
16	1.00	1.08	1.08	1.28

TABLE III

WHERE THE CYCLES GO AT $N=16$ WITH A VARYING ZERO POINT. CYCLES ARE THE SUM OF THE OTHER COLUMNS PLUS THE FIVE-STAGE DEPTH, RECOMPUTED FOR EVERY CASE RATHER THAN RECORDED. D FLUSHES NOTHING BECAUSE IT HAS NO LOOP TO CLOSE.

	Cycles	Retired	Load-use	Flushes
D	581	514	64	0
B3	745	624	88	15
B2	3926	3829	64	15
B1	19813	19716	64	15

D and B3 as 110 retired instructions, 24 stall cycles and 30 cycles of flush.

The flush column repays careful reading, because it is not a virtue. D flushes nothing at sixteen rows only because it has no back edge to take: its zero point is an immediate, so the rows are straight-line code. The fifteen flushes B3 pays are the price of keeping a loop, and the same choice is what keeps its code small. This is the structural difference the twin measurement exists to separate from the arithmetic one.

Under a varying zero point the raw gap mixes two effects, and the structural twin separates them. Of the 164 cycles that separate the variants at $N=16$, about 47 belong to loop control and 117 to the correction itself. The corrected ratio moves from 1.38 to 1.20 rather than from 1.46 to 1.28. Reporting only the raw figure would credit the instruction with work that the loop structure did.

Against scalar MUL, D uses between 5.4 and 6.8 times fewer cycles at $N=16$, the low end at $z=0$ where D has no correction to save; B1, without a multiplier, spends between 26 and 34 times more than D.

Specialization is not free, and the cost lands in a different currency. Table IV gives the full picture, which is less one-sided than a single pair of numbers suggests. Under a constant zero point D is the *smaller* of the two at any row count, 39 words against 68, because it needs no activation sum and no correction code and can keep a loop. It is only when the zero point varies that its text grows with the row count, reaching 516 words against 72, since the immediate demands one row body per distinct value.

TABLE IV

CODE SIZE IN INSTRUCTION WORDS. UNDER A CONSTANT ZERO POINT D IS SMALLER THAN B3 AND STAYS CONSTANT; UNDER A VARYING ONE IT GROWS LINEARLY WITH THE ROW COUNT WHILE B3 DOES NOT MOVE.

BOTH EFFECTS FOLLOW FROM THE SAME IMMEDIATE.

N	ZC $z=8$		ZS	
	D	B3	D	B3
1	39	68	39	68
4	39	68	132	72
16	39	68	516	72

The cycle advantage and the code-size penalty are two faces of one interface decision. Putting the zero point in an immediate is what makes the correction free at run time, what removes the loop, and what makes the text grow. A design that reported only the favorable half of that would be describing a different instruction.

Read together, these numbers answer the question with a condition rather than a verdict. Fusion helps most where the correction cannot be amortized: few rows and a non-zero zero point. It helps least, or not at all, where the alternative can either spread the correction across many rows or skip it entirely. And where it helps most in cycles under a varying zero point, it costs most in code.

A. Threats to Validity

The comparison is against baselines built for this study, not against the published extensions in Section II. Those target wider vectors, different precisions and whole convolution kernels, so a cycle count taken here would not be commensurable with one taken there; the consequence is that this work establishes where fusion helps relative to its own alternatives and not where it stands against the state of the art. The shared skeleton reduces optimization bias but does not establish optimality or remove the interaction between software and architecture. The synthetic distribution does not necessarily represent a real network. With $z=0$ the separate correction can be omitted, which does not establish equal performance in general. Dispatching among immediates would support a dynamic zero point without a new encoding, but has no implementation and no measured cost here. The counters are validated structurally, against eight directed programs, three negative controls, four detected harness mutations and a cycle accounting identity checked on every case; RTL cycles still do not establish physical timing. The sensitivity question is answered for the row count alone: one group of 32 elements was measured, so how the comparison moves with the group

count and the group size remains open, and the frozen data layout does not currently reach those shapes.

VI. CONCLUSION

Fusing the zero-point correction into the dot-product instruction reduces cycles against both factored correction and scalar multiplication, but not uniformly. The benefit narrows as the row count grows, disappears when the zero point is zero, and, where the zero point varies, is charged back in code size. The answer to the question is not a number but a map of conditions, and that map is what this work contributes.

Left outside are the regime with zero points read at run time, which this interface cannot express, and physical evidence: area, frequency and energy need synthesis and a different methodology. The additional hardware that the fused variant requires is not accounted for in any figure reported here.

REFERENCES

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," arXiv:1712.05877, 2017. [Online]. Available: <https://arxiv.org/abs/1712.05877>
- [2] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," arXiv:2103.13630, 2021. [Online]. Available: <https://arxiv.org/abs/2103.13630>
- [3] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "QServe: W4A8KV4 quantization and system co-design for efficient LLM serving," arXiv:2405.04532v3. <https://arxiv.org/abs/2405.04532v3>, 2025, primera versión: 2024; revisión consultada: 1 de mayo de 2025.
- [4] A. Garofalo, G. Tagliavini, F. Conti, L. Benini, and D. Rossi, "XpulpNN: Enabling energy efficient and flexible inference of quantized neural network on RISC-V based IoT end nodes," arXiv:2011.14325, 2020. [Online]. Available: <https://arxiv.org/abs/2011.14325>
- [5] G. Ottavi, A. Garofalo, G. Tagliavini, F. Conti, A. D. Mauro, L. Benini, and D. Rossi, "Dustin: A 16-cores parallel ultra-low-power cluster with 2b-to-32b fully flexible bit-precision and vector lockstep execution mode," arXiv:2201.08656v2, 2023. [Online]. Available: <https://arxiv.org/abs/2201.08656v2>
- [6] T. Dupuis, Y. Fournier, M. AskariHemmat, N. E. Zarif, F. Leduc-Primeau, J. P. David, and Y. Savaria, "Sparq: A custom RISC-V vector processor for efficient sub-byte quantized inference," arXiv:2306.09905, 2023. [Online]. Available: <https://arxiv.org/abs/2306.09905>
- [7] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: M extension, version 2.0," ISA v20260120. <https://docs.riscv.org/reference/isa/v20260120/unpriv/m-st-ext.html>, 2026, accessed: 2026-09-08.
- [8] —, "The RISC-V Instruction Set Manual, Volume I: C extension for compressed instructions, version 2.0," <https://docs.riscv.org/reference/isa/unpriv/c-st-ext.html>, 2026, biblioteca de especificaciones; fecha de consulta: 8 de septiembre de 2026. El año identifica la consulta, no la introducción de C.